

To run simulations using Logic Modeling Swift models with VCS, the following steps are taken:

- * Set LMC_HOME environment variable
- * Set LD_LIBRARY_PATH(sun) or LPATH/SHLIB_PATH(hp) environment variables
- * Set VCS_SWIFT_NOTES environment variable (optional)
- * Generate hdl templates:
% vcs -lmc-swift-template jkflipflop
- * Compile with vcs:
% vcs -lmc-swift testJkFlipFlop.v jkflipflop.swift.v
- * Run your simulation:
% simv

More details are provided below:

- * Make sure the appropriate LMC environment variables are set:
 - LMC_HOME must be set to the Swift release directory (containing the 'templates' directory).
 - For the SunOS platform, set environment variable LD_LIBRARY_PATH to include \${LMC_HOME}/lib/sun4SunOS.lib:
setenv LD_LIBRARY_PATH \${LD_LIBRARY_PATH}:\${LMC_HOME}/lib/sun4SunOS.lib
or, if LD_LIBRARY_PATH is not yet set:
setenv LD_LIBRARY_PATH \${LMC_HOME}/lib/sun4SunOS.lib
 - For the HP platform, set both environment variables LPATH and SHLIB_PATH:
setenv LPATH \${LPATH}:\${LMC_HOME}/lib/hp700.lib
setenv SHLIB_PATH \${SHLIB_PATH}:\${LMC_HOME}/lib/hp700.lib
or, if the variables are not yet set:
setenv LPATH \${LMC_HOME}/lib/hp700.lib
setenv SHLIB_PATH \${LMC_HOME}/lib/hp700.lib
 - For more verbose information from the swift interface:
setenv VCS_SWIFT_NOTES 1
This is usually recommended.

It is most convenient to set these variables in your .cshrc file.

- * Generate hdl templates for the swift models you will be using:

```
% vcs -lmc-swift-template jkflipflop
```

This will generate hdl file 'jkflipflop.swift.v'.
(Note that the vcs script simply invokes template generator '<vcs-release>/lmc/swift/vcsSwiftTemplate'; you can also use this program directly.)

You may want to alter the configuration of the port list in this generated file. You are allowed to alter the order of ports, you can concatenate ports, and you can explicitly name a port. Do not change the port names, and do not change the arguments to system task '\$vcs_swift'.

For example, given ports in generated hdl file 'zyx.swift.v':

```
module xyz (  
    I00, I01, I02, I03, ...
```

You can reorder:

```
module xyz (  
    I03, I02, I01, I00, ...
```

You can concatenate:

```
module xyz (  
    {I03, I02, I01, I00}, ...
```

This allows you to then conveniently connect vector ports:

```
wire [3:0] bus4;  
...  
xyz xyz1( bus4, ...
```

Further, you can explicitly name:

```
module xyz (  
    .IO({I03, I02, I01, I00}), ...
```

This allows you to then conveniently connect vector ports by name:

```
wire [3:0] bus4;  
...  
xyz xyz1( ..., .IO(bus4), ...
```

The hdl template file will also contain several parameters representing swift model attributes. For example, a swift model 'mem' may contain attribute 'memoryFile', which will appear in the 'mem.swift.v' file as:

```
parameter MemoryFile = "memory";
```

If you know that all instances of this model will need to load memory file 'mem.dat', you can change this to:

```
parameter MemoryFile = "mem.dat";
```

Otherwise, you can deal with this on an instance by instance basis with defparams:

```
mem mem1( a, ce, io, oe, we);  
defparam mem1.MemoryFile = "mem1.dat";
```

You can also change the name of the parameters in the template module. This is particularly useful if you have existing models which instantiated smartmodels, since the smartmodel interace used different parameter names. For example, what smartmodels called 'COMPONENT', swift models refer to as 'TimingVersion'. You can alter the template as follows if your hdl contains defparams of 'COMPONENT':

Change the parameter definition name:

```
parameter TimingVersion = "BUFF-0";
```

To:

```
parameter COMPONENT = "BUFF-0";
```

Change the parameter usage, but **not** the associated string,
in the \$vcs_swift system task call:

```
$vcs_swift("buff",  
    ...  
    "attr", "TimingVersion", TimingVersion,  
To:  
    $vcs_swift("buff",  
    ...  
    "attr", "TimingVersion", COMPONENT,
```

A defparam would then change from:

```
BUFF g1(a,q);  
defparam g1.TimingVersion = "BUFF-1"  
To:  
BUFF g1(a,q);  
defparam g1.COMPONENT = "BUFF-1"
```

* Run vcs with the '-lmc-swift' switch:

```
% vcs -lmc-swift testJkFlipFlop.v jkflipflop.swift.v -o jkff
```

This will produce simulation executable 'jkff' (or 'simv' if
-o flag is not used).

* Simulate:

```
% jkff
```

(For more verbose information, including versions, set environment
variable VCS_SWIFT_NOTES to 1.)

A mechanism to assess the performance overhead of the swift interface
and models is also available. To use it, edit your 'csrc/Makefile' and
change '.../vcsSwift.o' to '.../vcsSwift.prof.o', delete the executable
('../simv' by default), and run make. When you run the resultant
executable, a banner will indicate that it is a profiling version,
and at the end of simulation a summary of swift usage is presented.

You can perform a dump of a swift memory by setting template module
variable 'mem\$dump\$file' to the desired file name, then setting
'do\$mem\$dump' to 1. For instance, if you had a memory file called
'top.asic.mem1' and you wished to dump to file 'mem1.lmc.1k.dump'
at time 1000, and 'mem1.lmc.2k.dump' at time 2000, you would add
the following code:

```
initial begin  
    #1000  
    top.asic.mem1.mem$dump$file = "mem1.lmc.1k.dump";
```

```
top.asic.mem1.do$mem$dump = 1;  
#1000  
top.asic.mem1.mem$dump$file = "mem1.lmc.2k.dump";  
top.asic.mem1.do$mem$dump = 1;  
end
```

You can access the swift model command channel in a similar manner by setting template variable 'cmd\$str' to the desired command, then setting either 'do\$model\$cmd' or 'do\$session\$cmd' to 1.